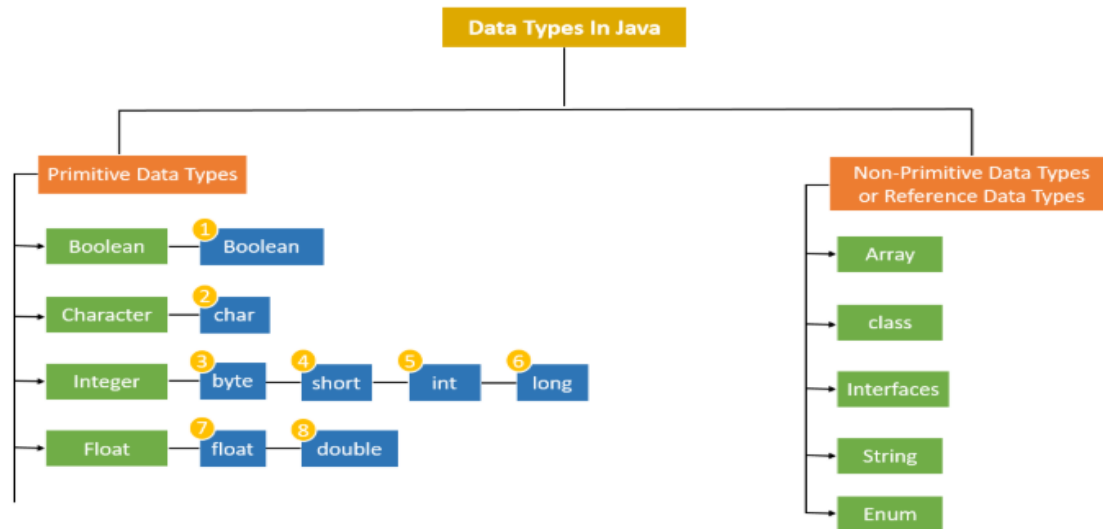


Primitive Data types, Variables

Data types in Java specify the size and type of values that a variable can store. Java is a **strongly typed language**, meaning every variable must be declared with a specific data type before it can be used. Java data types are divided into two main categories: **Primitive** and **Non-Primitive**.



TYPE	DESCRIPTION	DEFAULT	SIZE	EXAMPLE LITERALS	RANGE OF VALUES
boolean	true or false	false	1 bit	true, false	true, false
byte	twos complement integer	0	8 bits	(none)	-128 to 127
char	unicode character	\u0000	16 bits	'a', '\u0041', '\101', '\w', '\l', '\ln', '\b'	character representation of ASCII values 0 to 255
short	twos complement integer	0	16 bits	(none)	-32,768 to 32,767
int	twos complement integer	0	32 bits	-2, -1, 0, 1, 2	-2,147,483,648 to 2,147,483,647
long	twos complement integer	0	64 bits	-2L, -1L, 0L, 1L, 2L	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	IEEE 754 floating point	0.0	32 bits	1.23e100f, -1.23e-100f, .3f, 3.14F	upto 7 decimal digits
double	IEEE 754 floating point	0.0	64 bits	1.23456e300d, -1.23456e-300d, 1e1d	upto 16 decimal digits

Numeric Types (Integers): Used to store whole numbers without decimals.

Data Type	Size	Memory Range	Common Use Case
<code>byte</code>	1 byte (8 bits)	-128 to 127	Saving memory in large arrays
<code>short</code>	2 bytes (16 bits)	-32,768 to 32,767	Rare; used for memory constraints
<code>int</code>	4 bytes (32 bits)	-2.14 Billion to 2.14 Billion	Default choice for whole numbers
<code>long</code>	8 bytes (64 bits)	-9 Quintillion to 9 Quintillion	Large numbers (IDs, system time)

Note: When assigning a value to a long, append an L or l to the end of the number (e.g., long population = 8000000000L;)

Numeric Types (Floating-Point): Used to store fractional numbers containing decimals.

Data Type	Size	Precision	Common Use Case
<code>float</code>	4 bytes (32 bits)	6 to 7 decimal digits	Game graphics, saving memory
<code>double</code>	8 bytes (64 bits)	Up to 15 decimal digits	Default choice for math and decimals

⚠ **Note:** Java treats all decimal numbers as `double` by default. To declare a `float`, you must append an `F` or `f` to the value (e.g., `float price = 19.99f;`). ☹

Character Type: Used to store a single character.

Data Type	Size	Contents	Description
<code>char</code>	2 bytes (16 bits)	Single character or ASCII	Uses single quotes (<code>'A'</code> , <code>'3'</code> , <code>'\$'</code>). Uses Unicode system.

Boolean Type: Used to store conditional states.

Data Type	Size	Allowed Values	Common Use Case
<code>boolean</code>	1 bit (virtual)	<code>true</code> or <code>false</code>	Conditional flags (<code>if/else</code> logic)

Variables

A **variable** is a named storage container in a computer's memory used to hold data that a program can manipulate and change during execution. Think of it as a labeled box on a shelf; the label is the variable's name, and the item inside the box is its value.

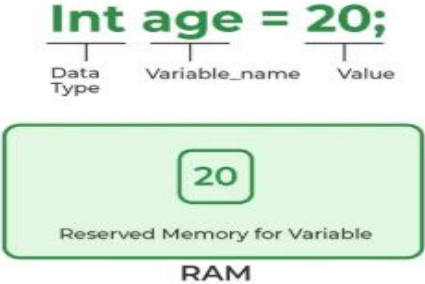
The Three Rules of Java Variables

Because Java is strongly typed, every variable must follow a strict three-part declaration syntax:

1. **Type:** What kind of data can fit inside the box (e.g., int, double, String).
2. **Name:** The identifier used to call upon that specific box in your code (e.g., age, price).
3. **Value:** The actual data stored inside the box (e.g., 25, 19.99).

Variable Syntax and Examples

You create a variable by **declaring** its type and name, and you populate it by **initializing** it with an assignment operator (=).

	<pre>// 1. Declaration only (Creating the empty box) int score; // 2. Initialization (Putting a value into the box) score = 10; // 3. Combined Declaration and Initialization (Most common) String username = "Alex"; // 4. Changing the value later (Overwriting the box contents) score = 15;</pre>
--	---

Variable Scopes in Java

Where you declare a variable determines where it can be used inside your code:

- **Local Variables:** Declared inside a method (like main). They only exist while that specific method is running and cannot be accessed from outside it.
- **Instance Variables:** Declared inside a class but outside any methods. They belong to objects created from the class.
- **Static Variables:** Declared with the static keyword inside a class. They belong to the class itself and are shared globally across all instances.

Rules for Naming Variables (Identifiers)

- Names are **case-sensitive** (age and Age are completely different boxes).
- Names must start with a letter, a dollar sign (\$), or an underscore (_). They cannot start with a number.
- Names cannot contain spaces.
- Names cannot use Java reserved keywords (you cannot name a variable public or class).
- Java convention uses **camelCase** for multi-word names

Reserved Words

Reserved words (also known as **keywords**) are predefined words that have a special, built-in meaning to the Java compiler. Because the language reserves these words for its own structural rules, logic, and commands, you **cannot** use them as names for your variables, classes, methods, or any other identifiers.

Java has over 50 reserved words. They are always written in lowercase.

Categories of Common Reserved Words

1. Data Types & Values

Words used to declare variable types or literal states.

- **int, double, char, boolean, long, float, byte, short:** Define primitive data types.
- **true, false:** Literal boolean values.
- **null:** Represents an empty or non-existent reference.

2. Access Modifiers

Words that control which parts of your program can see or use a class, method, or variable.

- **public:** Accessible from anywhere in your project.
- **private:** Accessible only inside the specific class where it was created.
- **protected:** Accessible within the same package or by child classes.

3. Flow Control & Logic

Words that dictate the order in which your code runs.

- **if, else:** Used to make conditional decisions.
- **switch, case, default:** Used for multi-way branching selections.
- **for, while, do:** Used to create loops that repeat code blocks.
- **break:** Exits a loop or switch block immediately.
- **continue:** Skips the rest of the current loop iteration and moves to the next one.
- **return:** Exits a method and optionally sends back a value.

4. Program Structure & Modifiers

Words used to build and organize the layout of your application.

- **class:** Declares a new object blueprint.
- **interface:** Declares a structural contract for classes.
- **package:** Groups related classes together into a folder system.
- **import:** Brings external classes or packages into your current code file.
- **static:** Attaches a method or variable directly to the class rather than an object instance.
- **void:** Tells a method that it will not return any data.
- **final:** Makes a variable constant (unchangeable) or prevents a class from being inherited.

❌ Invalid vs. ✅ Valid Variable Names

java

```
// INVALID CODE - Will cause a compiler error:
int public = 25;           // Error: 'public' is a reserved word
double class = 5.99;       // Error: 'class' is a reserved word
boolean void = true;       // Error: 'void' is a reserved word

// VALID CODE - Safe to use:
int publicScore = 25;      // OK: Combines keyword with another word
double classPrice = 5.99;  // OK: Modified name
boolean isValid = true;    // OK: Plain English word (not reserved)
```